

Esercizio di programmazione concorrente

Si consideri il codice dato, che simula un gioco, in cui compaiono dei thread Giocatore e un TavoloGioco condiviso tra i giocatori.

Ad ogni turno, un giocatore ne attacca un altro, che risponde: pertanto, ad ogni turno si svolgono due mosse, una di attacco e una di difesa.

Ciascun giocatore agisce come segue:

- 1) Si aggiunge al tavolo da gioco, ricevendo un identificatore numerico (il primo giocatore che arriva sarà giocatore_0, il secondo sarà giocatore_1, e così via).
- 2) Aspetta che si siano aggiunti al tavolo tanti giocatori quanti richiesti dalle regole del gioco.
- 3) Gioca. Un giocatore effettua una mossa in due occasioni: quando è il suo turno oppure quando viene attaccato da un altro giocatore. Dunque il comportamento del giocatore comprende una serie di iterazioni, in cui
 - a) attende
 - b) se la partita è finita termina, altrimenti effettua la sua mossa (di difesa o di attacco secondo i casi).

I dettagli del gioco e delle mosse sono irrilevanti.

Il gestore del tavolo determina quale giocatore è di turno: la sequenza dei turni sarà giocatore_0, giocatore_1, giocatore_2, ecc. Quando il giocatore di turno effettua una mossa di attacco, la mossa successiva spetta al giocatore attaccato; dopo di che, si riprende con i turni normali. Ad es., giocatore_2 è di turno e attacca giocatore_1, quindi giocatore_1 fa la sua mossa difensiva; a questo punto il turno è finito e tocca a giocatore_3, e così via.

Il codice dato non comprende la sincronizzazione: si chiede pertanto di modificarlo in modo che il gioco e le mosse si svolgano come descritto sopra, evitando i problemi tipici della programmazione concorrente (corse critiche, deadlock, ecc.). NB: per ottenere il comportamento desiderato è sufficiente modificare la classe TavoloGioco; tuttavia, chi volesse è libero di fare scelte diverse, purché il comportamento risultante sia quello prescritto.

Si ricorda che bisogna consegnare i file .java (non i .class!), raggruppati in un unico file zip.

Esercizio 2. Programmazione distribuita / socket

Si chiede di trasformare il programma sviluppato come soluzione dell'esercizio di programmazione concorrente in un sistema distribuito, in cui il server gestisce il gioco e i client operano come giocatori.

Si realizzi il sistema mediante l'uso di socket.

Come sempre, occorre evitare i problemi tipici della programmazione concorrente (corse critiche, deadlock, starvation, ecc.).

Si ricorda che bisogna consegnare i file .java (non i .class!), raggruppati in un unico file zip.

Esercizio di programmazione distribuita (con uso di RMI)

Si chiede di trasformare il programma sviluppato come soluzione dell'esercizio di programmazione concorrente in un sistema distribuito, in cui il server gestisce il gioco e i client operano come giocatori.

Si realizzi il sistema mediante l'uso di RMI.

Come sempre, occorre evitare i problemi tipici della programmazione concorrente (corse critiche, deadlock, starvation, ecc.).

Si ricorda che bisogna consegnare i file .java (non i .class!), raggruppati in un unico file zip.